LAB 1 · BÀI TẬP & BÁO CÁO

Báo cáo LAB 1 — Làm quen với CoreIOT

Năm bài tập cơ bản: thiết bị, telemetry, dashboard, điều khiển RPC và chia sẻ

Họ và tên	Phạm Công Võ
MSSV	2313946
Lớp / nhóm	L01
Ngày	24/10/2026
Tên thiết bị trên CoreIOT	Lab1_2313946
Link dashboard công khai	https://app.coreiot.io/dashboard/90b3bff0-cef3-11f1-9298-f795494fee72?publicId=a5c11a80-a4e4-11f0-b5f4-25fce636d3ff

Cách hoàn thành báo cáo này

- Làm theo **hướng dẫn LAB 1** (Làm quen với CoreIOT) trong khi thực hành; các số mục như "Hướng dẫn 4.3" là tham chiếu đến tài liệu đó.
- **Các thao tác trên CoreIOT** (thiết bị, widget, dashboard): chụp màn hình và dán vào khung nét đứt. Đảm bảo **tên thiết bị** của bạn hiển thị trong ít nhất một ảnh chụp màn hình của mỗi bài tập.
- **Mã Python:** dán vào khung mã màu xám bằng **Ctrl+Shift+V** (dán dạng văn bản thuần) để giữ nguyên thụt lề. Dùng font `Courier New` nếu bạn tự gõ mã.
- Trả lời mỗi câu hỏi bằng lời của chính bạn, trong 2–4 câu.
- Dùng tên thiết bị `LAB1_<StudentID>` cho tất cả các bài tập, ví dụ `LAB1_2212345`.

Tuyệt đối không nộp access token của bạn

Trước khi dán mã hoặc ảnh chụp màn hình, hãy thay token bằng `***` hoặc làm mờ nó. Token bị lộ cho phép bất kỳ ai gửi dữ liệu dưới danh nghĩa thiết bị của bạn.

Bài tập	Chủ đề	Minh chứng	Điểm
1	Thiết bị, thông tin xác thực và nhóm	3 ảnh chụp màn hình + 1 câu trả lời	1.5
2	Telemetry từ Python (thêm cảm biến ánh sáng)	Mã + 2 ảnh chụp màn hình + 1 câu trả lời	2.0
3	Dashboard: thẻ giá trị và biểu đồ	2 ảnh chụp màn hình + 1 câu trả lời	2.5
4	Điều khiển RPC: LED, Bơm và công tắc Quạt mới	Mã + 3 ảnh chụp màn hình + 1 câu trả lời	2.5
5	Chia sẻ dashboard công khai	Link + 2 ảnh chụp màn hình + 1 câu trả lời	1.5
	Tổng		**10**

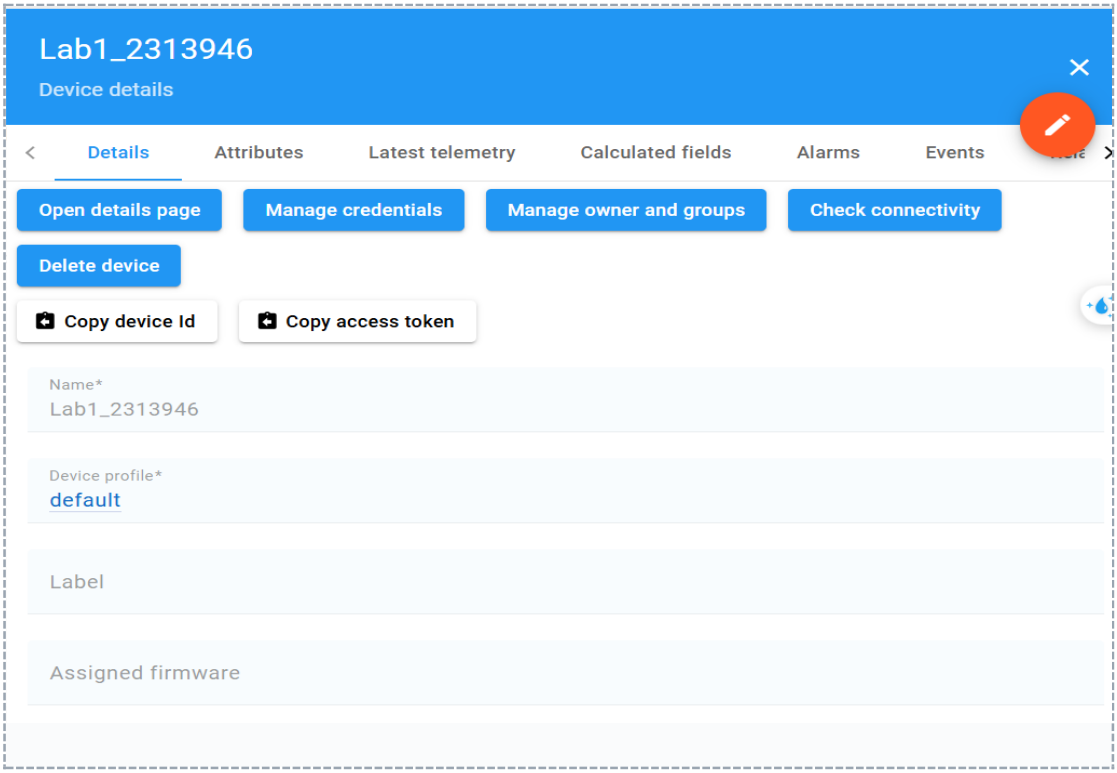
BÀI TẬP 1 **Tạo thiết bị và thông tin xác thực** CoreIOT · ảnh chụp màn hình **1,5 điểm**

Yêu cầu

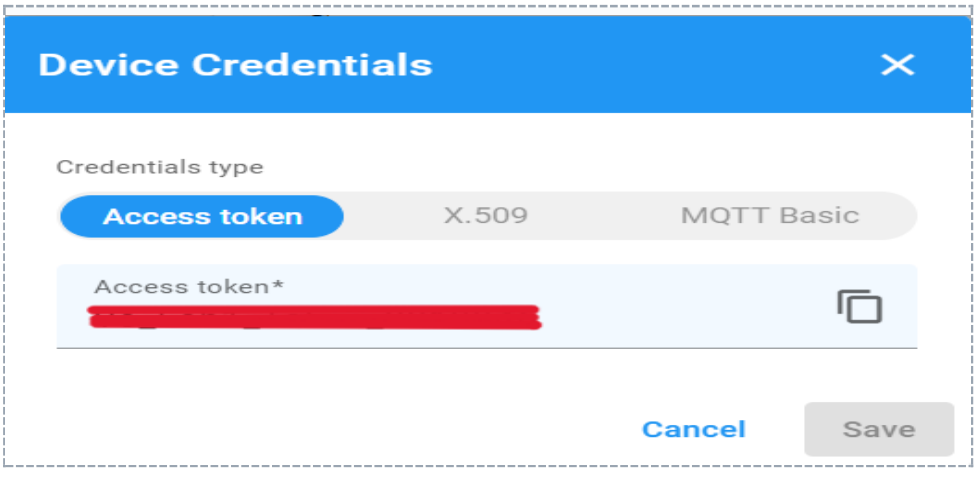
1. Tạo một thiết bị tên `LAB1_<StudentID>` với device profile `default` (Hướng dẫn 4.1).
2. Mở **Manage credentials** và đặt một **access token tùy chỉnh** do bạn chọn (ít nhất 12 ký tự). Lưu lại (Hướng dẫn 4.3).
3. Tạo một nhóm thiết bị tên `LAB1_<StudentID>_group` và thêm thiết bị của bạn vào nhóm (Hướng dẫn 4.4).

Minh chứng

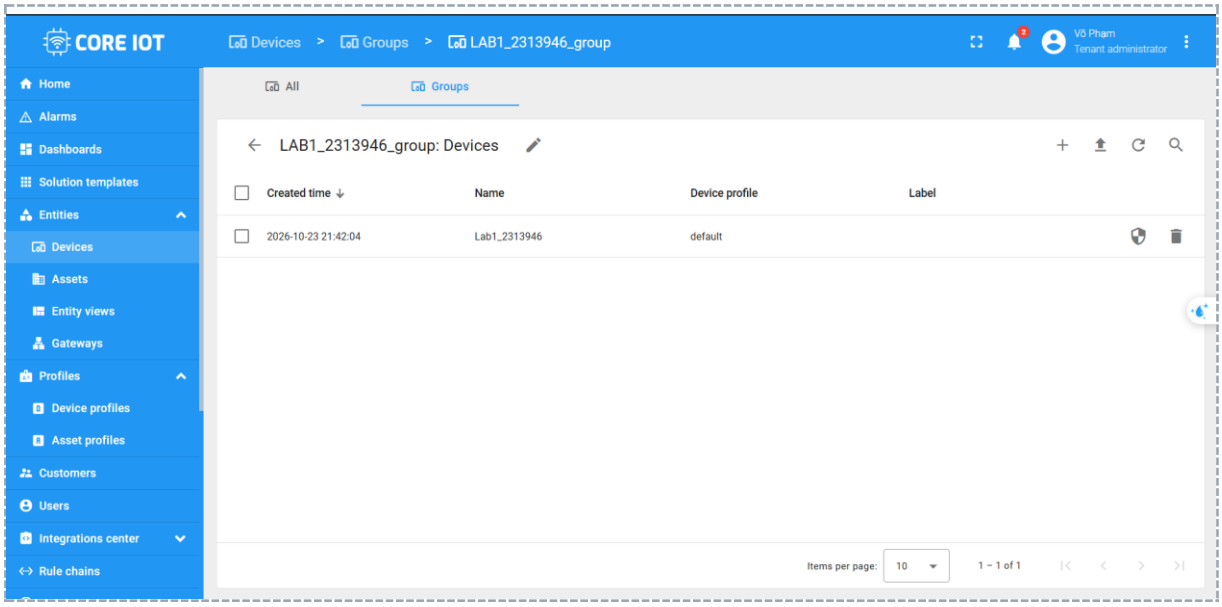
Ảnh 1.1. Trang chi tiết thiết bị của bạn (tab **Details**).



Ảnh 1.2. Hộp thoại **Device Credentials** với loại thông tin xác thực được đặt là Access token.



Ảnh 1.3. Nhóm thiết bị của bạn khi được mở, hiển thị thiết bị của bạn bên trong.



Câu hỏi 1. Sự khác nhau giữa hai loại thông tin xác thực **Access token** và **MQTT Basic** là gì? Vì sao phải giữ bí mật token?

- Khác nhau: Access token sử dụng một chuỗi bí mật duy nhất được gửi làm username của MQTT (bỏ trống trường password), giúp cơ chế kết nối cực kỳ nhanh gọn và tối ưu cho thiết bị IoT bị giới hạn tài nguyên. Trong khi đó, MQTT Basic yêu cầu cấu hình đầy đủ bộ ba: Client ID, User Name và Password.

- Vì sao phải giữ bí mật: Access token chính là "chìa khóa định danh" độc quyền của thiết bị trên CoreIOT. Nếu token bị lộ, kẻ xấu có thể kết nối từ xa để giả mạo thiết bị, bơm dữ liệu rác làm sai lệch hệ thống giám sát hoặc chiếm quyền gửi lệnh điều khiển phần cứng.

BÀI TẬP 2 **Gửi telemetry từ Python** Python · mã + ảnh chụp màn hình **2,0 điểm**

Yêu cầu

1. Bắt đầu từ `lab1_device.py` (Hướng dẫn 5.2) và đặt access token của bạn.
2. Thêm khóa telemetry thứ ba tên `light` với giá trị **số nguyên** ngẫu nhiên từ 0 đến 1000 (gợi ý: `random.randint(0, 1000)`).
3. Đổi chu kỳ gửi từ 5 s thành **3 s**.
4. Chạy script ít nhất một phút.

Minh chứng

Mã 2.1 · vòng lặp gửi telemetry trong script của bạn (khối `while True:` và mọi phần bạn đã thay đổi)

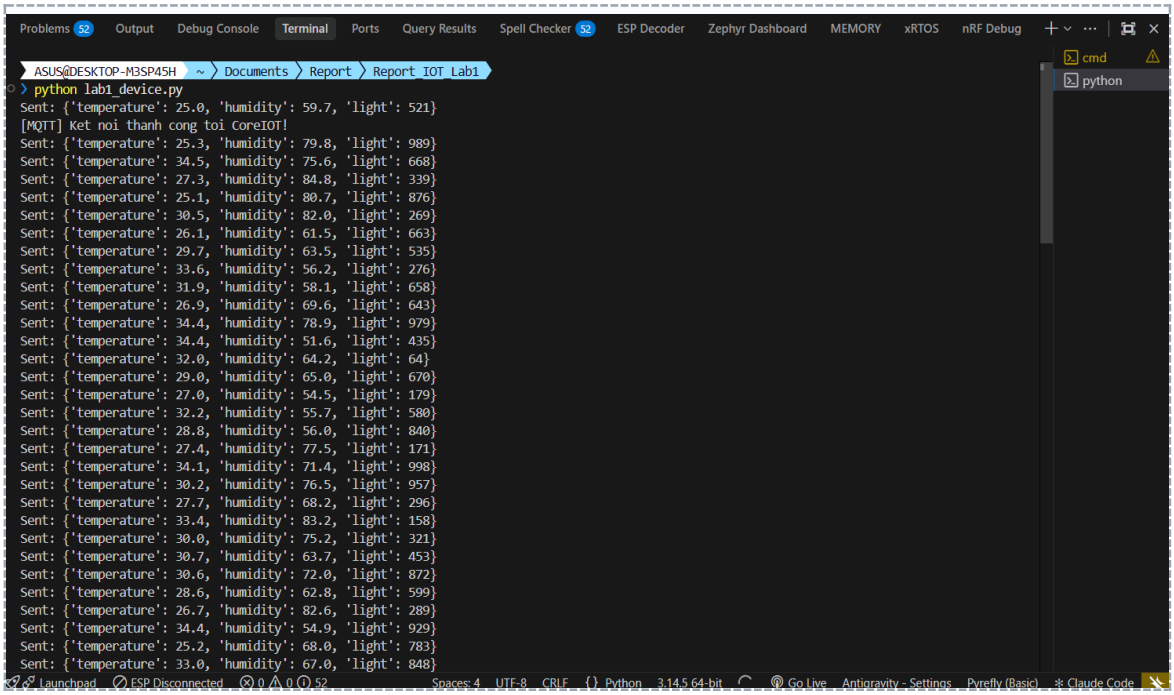
```
try:
    while True:
        # Yêu cầu Bài 2: nhiệt độ, độ ẩm và cảm biến ánh sáng
        temp = round(random.uniform(25.0, 35.0), 1)
        humi = round(random.uniform(50.0, 85.0), 1)
        light = random.randint(0, 1000)

        payload = {
            "temperature": temp,
            "humidity": humi,
            "light": light
        }
        client.publish("v1/devices/me/telemetry",
                       json.dumps(payload), qos=1)
        print(f"Sent: {payload}")

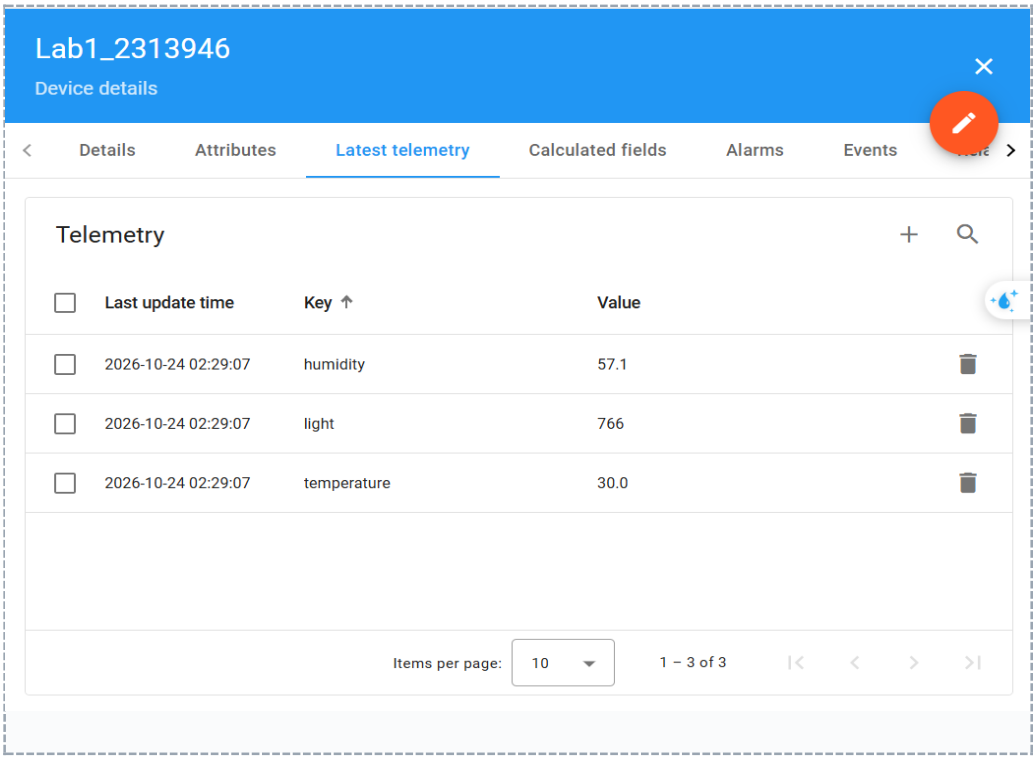
        # Yêu cầu Bài 2: Đổi chu kỳ gửi thành 3 giây
        time.sleep(3)

except KeyboardInterrupt:
    print("\nDang ngat ket noi...")
```

Ảnh 2.1. Kết quả trên terminal hiển thị `Connected to CoreIOT` và một số dòng `Sent:` có chứa `light`.



Ảnh 2.2. Tab **Latest telemetry** của thiết bị hiển thị `temperature`, `humidity` và `light`.



Câu hỏi 2. Script gửi telemetry lên MQTT topic nào, và một bản tin trông như thế nào (viết một ví dụ payload JSON)?

- Script gửi telemetry lên MQTT topic chuẩn: **v1/devices/me/telemetry**

- Bản tin được đóng gói dưới dạng chuỗi JSON gồm 3 cặp khóa-giá trị, ví dụ: **{"temperature": 29.4, "humidity": 68.5, "light": 650}**

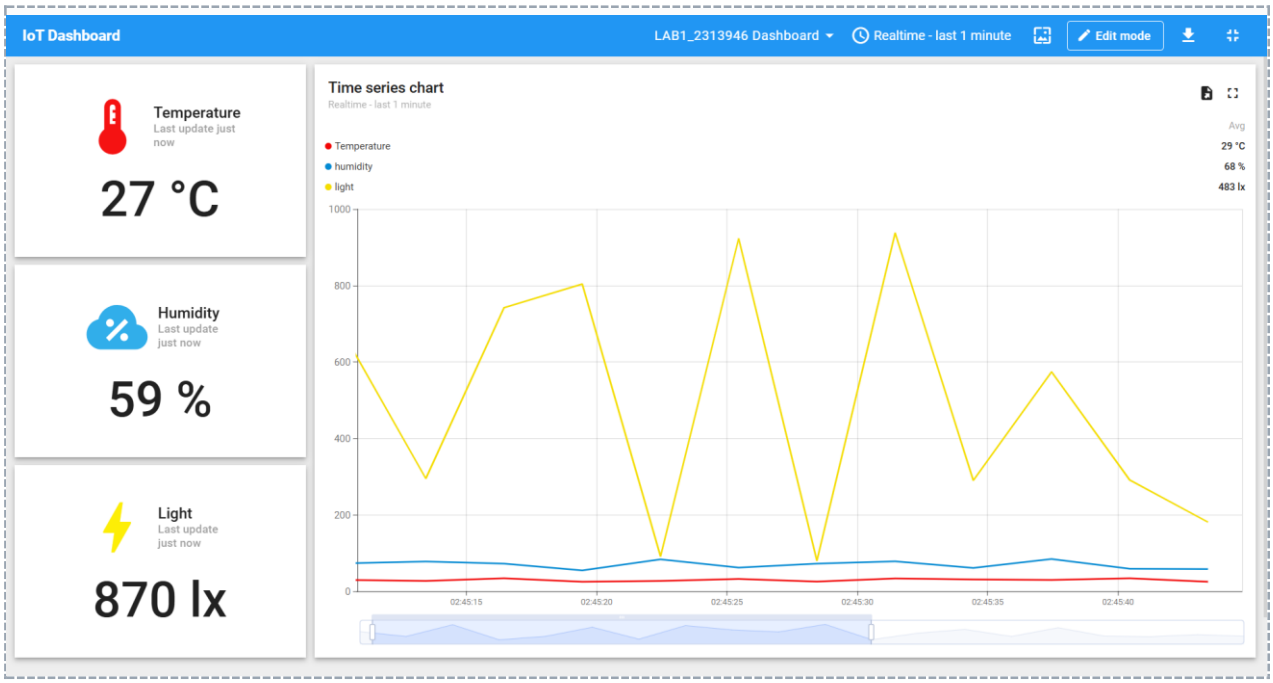
BÀI TẬP 3 **Xây dựng dashboard giám sát** CoreIOT · ảnh chụp màn hình **2,5 điểm**

Yêu cầu

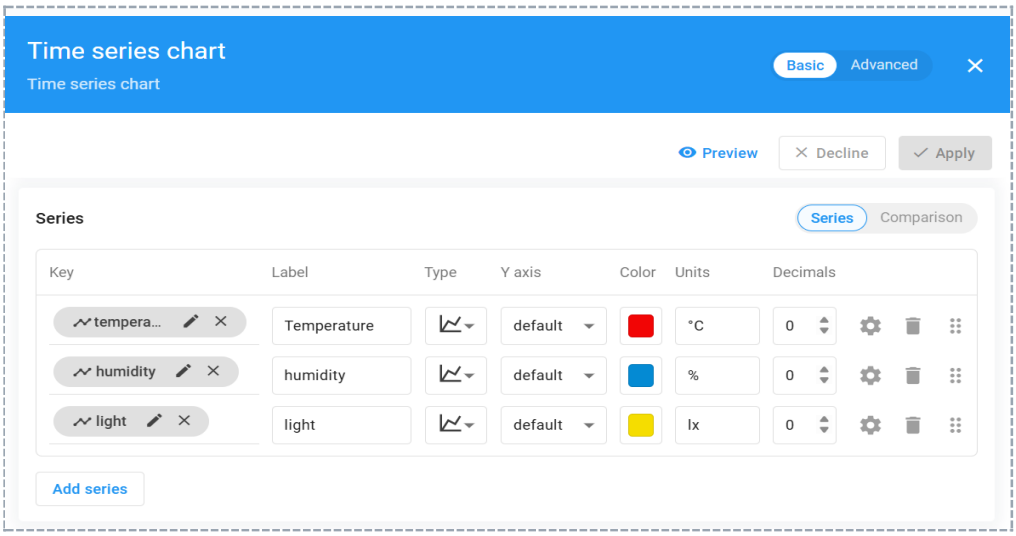
1. Tạo một dashboard tên `LAB1_<StudentID> Dashboard` (Hướng dẫn 6.1).
2. Thêm ba widget **Value card**: Nhiệt độ (°C), Độ ẩm (%) và Ánh sáng (lux), mỗi widget có nhãn, đơn vị và biểu tượng phù hợp (Hướng dẫn 6.2–6.3).
3. Thêm một **Time series chart** hiển thị cả ba khóa dưới dạng các chuỗi dữ liệu (series) riêng biệt (Hướng dẫn 6.6).
4. Sắp xếp các widget gọn gàng và nhấn **Save**.

Minh chứng

Ảnh 3.1. Toàn bộ dashboard (chế độ xem) với ba thẻ và biểu đồ hiển thị dữ liệu trực tiếp.



Ảnh 3.2. Trình chỉnh sửa widget của biểu đồ chuỗi thời gian, hiển thị ba series trong danh sách **Series**.



Câu hỏi 3. Khi bạn thêm Value card lần đầu, vì sao khóa `light` có thể không xuất hiện trong danh sách data key? Bạn khắc phục bằng cách nào?

> **- Nguyên nhân:** Nền tảng CoreIOT chỉ tự động thu thập và hiển thị tên khóa (data key) trong menu gợi ý sau khi máy chủ nhận được ít nhất một bản tin telemetry chứa khóa đó từ thiết bị gửi lên. Nếu script chưa chạy hoặc chưa gửi gói tin nào có trường light, máy chủ sẽ chưa biết đến khóa này.
>
> **- Khắc phục:** Đảm bảo script Python đã chạy ít nhất 1 chu kỳ để gửi khóa light lên trước khi thêm widget; hoặc có thể gõ trực tiếp chữ light vào ô tìm kiếm data key rồi nhấn Enter để ép chọn.

BÀI TẬP 4 **Điều khiển từ xa bằng RPC** CoreIOT + Python **2,5 điểm**

Yêu cầu

1. Thêm các widget Single Switch **LED** và **Pump** (Bơm) vào dashboard (Hướng dẫn 6.4–6.5).
2. Thêm công tắc thứ ba **Fan** (Quạt) sử dụng các phương thức RPC `getStateFAN` và `setStateFAN`.
3. Cập nhật script để xử lý thêm Quạt: script phải ghi nhớ trạng thái Quạt, trả lời `getStateFAN` và in ra thông báo rõ ràng mỗi khi có đầu ra thay đổi, ví dụ `LED -> ON` hoặc `FAN -> OFF`.
4. Bật/tắt mỗi công tắc ít nhất một lần, tải lại dashboard và kiểm tra rằng các công tắc vẫn giữ nguyên vị trí.

Minh chứng

Mã 4.1 · dictionary `state` và toàn bộ hàm `on_message()` của bạn

```
# Trạng thái ban đầu của 3 thiết bị chấp hành
state = {"LED": False, "PUMP": False, "FAN": False}

def on_connect(client, userdata, flags, reason_code, properties):
    if reason_code == 0:
        print("[MQTT] Ket noi thanh cong toi CoreIOT!")
        # Đăng ký nhận lệnh RPC từ Dashboard
        client.subscribe("v1/devices/me/rpc/request/+")
    else:
        print(f"[MQTT] Ket noi that bai, reason
              code:{reason_code}")


def on_message(client, userdata, msg):
    request_id = msg.topic.split("/")[-1]
    try:
        req = json.loads(msg.payload)
    except json.JSONDecodeError:
        print("[RPC] Payload JSON khong hop le:", msg.payload)
        return

    method = req.get("method", "")
    params = req.get("params")
    result = None

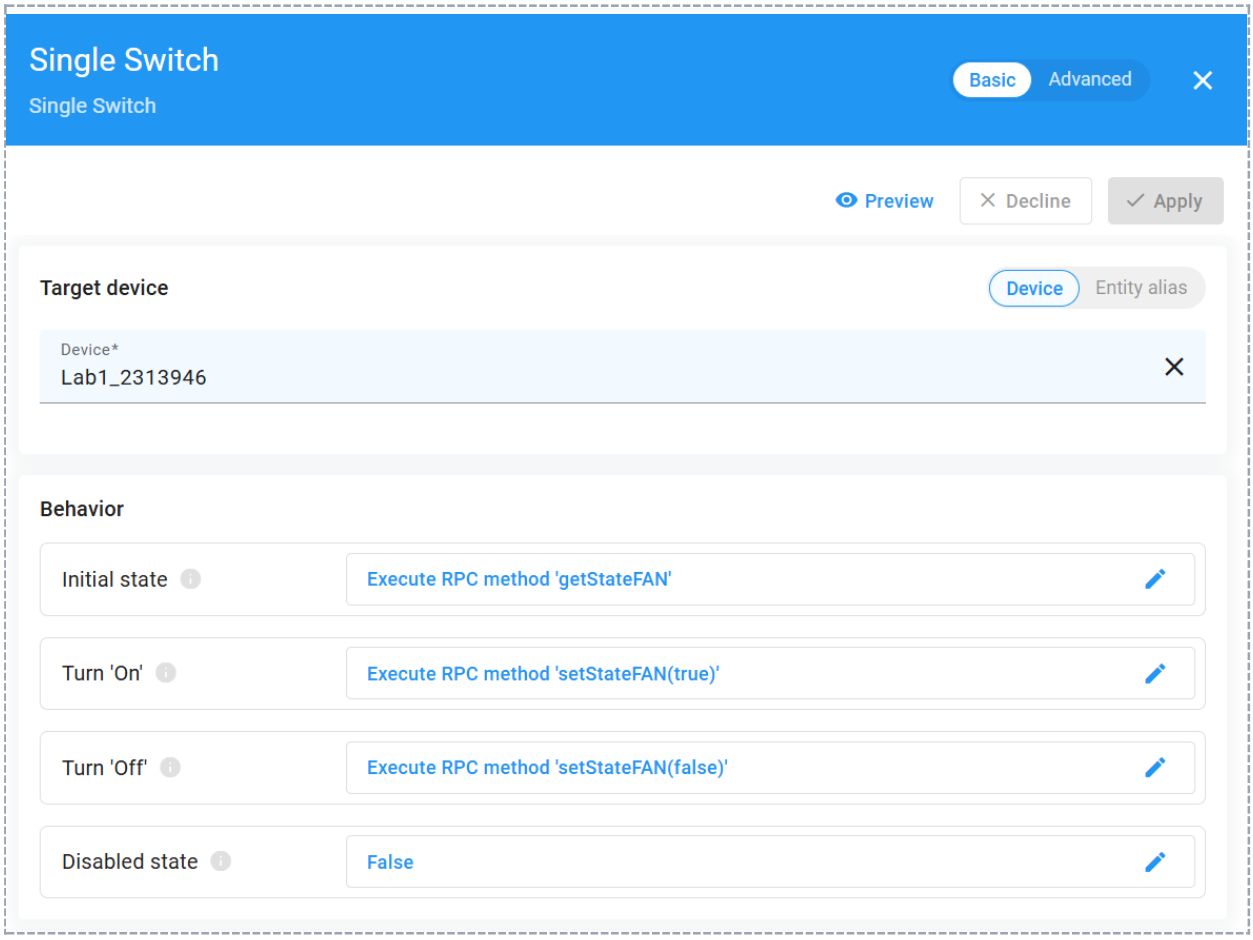
    # Xử lý getState
    if method.startswith("getState"):
        name = method[len("getState"):]
        result = state.get(name, False)

    # Xử lý setState (Nhận lệnh bật/tắt từ công tắc Dashboard)
    elif method.startswith("setState"):
        name = method[len("setState"):]
        if name in state:
            state[name] = bool(params)
            result = state[name]
```

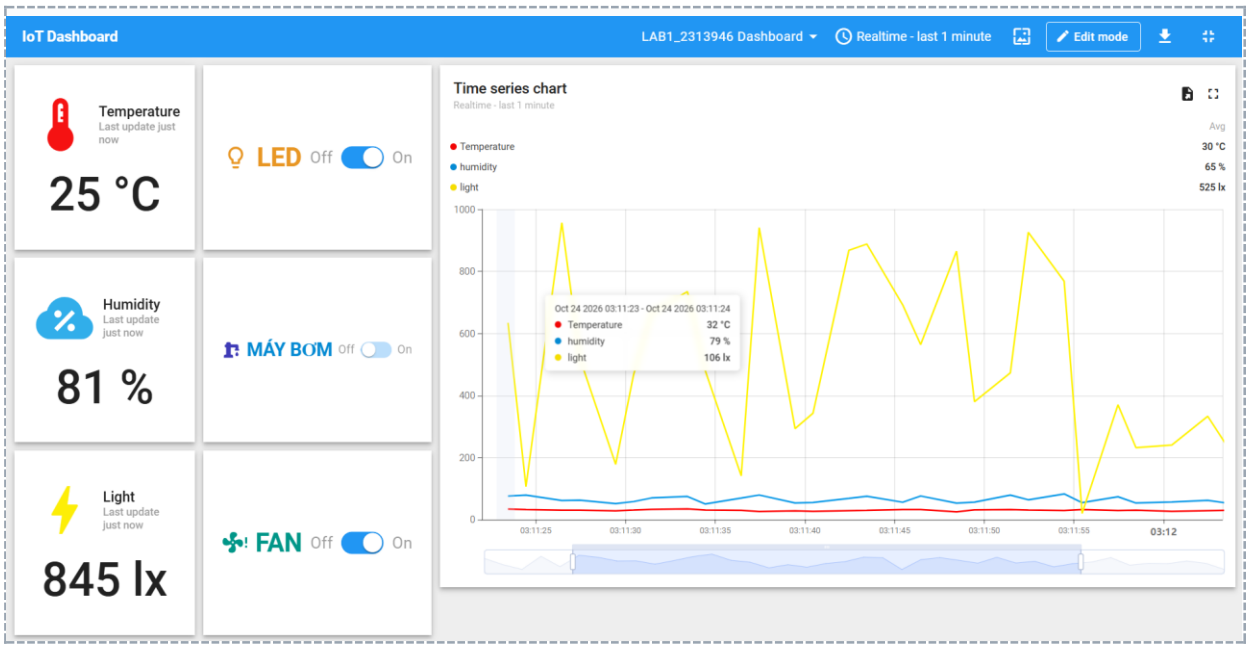
```
            # In thông báo trạng thái rõ ràng ra terminal
            status_str = "ON" if result else "OFF"
            print(f"[DIEU KHIEN] {name} -> {status_str}")
            # Cập nhật client attribute lên CoreIOT
            client.publish("v1/devices/me/attributes",
                  json.dumps({name.lower() + "_state": result}))

    # Phản hồi bắt buộc về topic response để Switch trên
Dashboard không bị Timeout
    client.publish(f"v1/devices/me/rpc/response/{request_id}",
            json.dumps(result), qos=1)
```

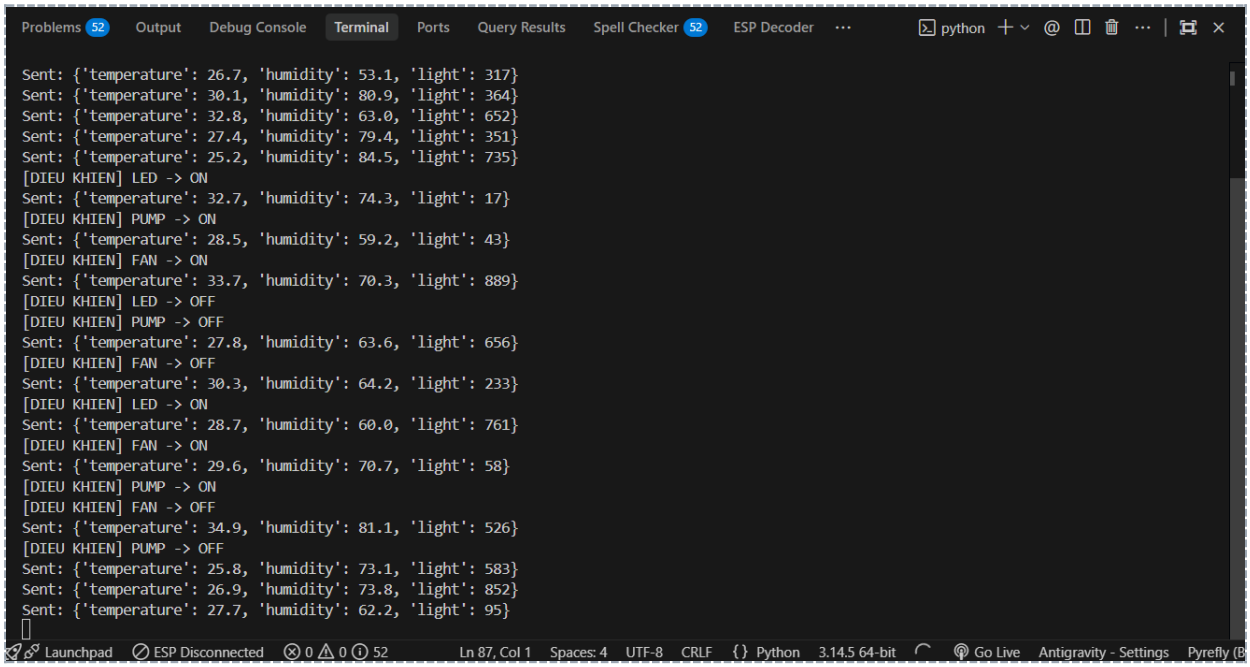
Ảnh 4.1. Phần **Behavior** trong trình chỉnh sửa công tắc Quạt, hiển thị ba phương thức RPC.



Ảnh 4.2. Dashboard với ba công tắc, sau khi bạn đã bật/tắt chúng.



Ảnh 4.3. Kết quả trên terminal với các yêu cầu RPC và các thông báo `-> ON` / `-> OFF` của bạn.



Câu hỏi 4. Dừng script và bật/tắt một công tắc. Widget hiển thị gì, và vì sao? Vì sao thiết bị phải gửi phản hồi tới `v1/devices/me/rpc/response/{id}`?

> **- Khi dừng script và gạt công tắc:** Widget sẽ quay tròn chờ đợi rồi báo lỗi **Failed to perform the " " action. Request timeout** sau đó công tắc tự nhảy ngược về vị trí cũ. Lý do vì thiết bị đã ngắt kết nối nên không nhận được lệnh và không có ai gửi gói tin phản hồi về server.
> **- Vì sao phải gửi phản hồi về topic response:** Đây là cơ chế gọi thủ tục hai chiều (Two-way RPC). Server CoreIOT gửi lệnh kèm một mã {id} và giữ kết nối chờ xác nhận. Thiết bị bắt buộc phải gửi kết quả trả lời về đúng topic v1/devices/me/rpc/response/{id} để máy chủ xác nhận lệnh đã được thực thi thành công dưới phần cứng, từ đó widget trên Dashboard mới chính thức xác nhận trạng thái mới.

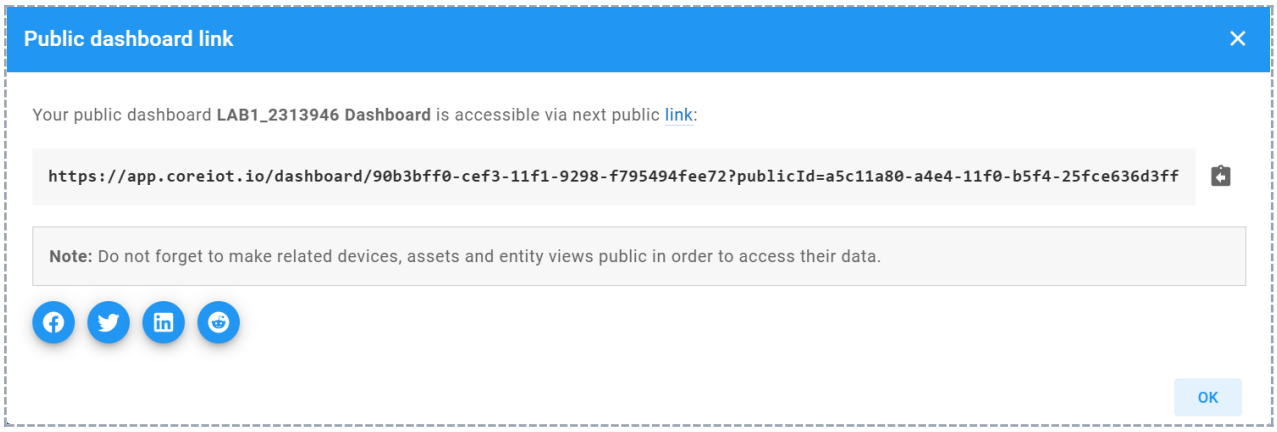
BÀI TẬP 5 **Chia sẻ dashboard** CoreIOT · link + ảnh chụp màn hình **1,5 điểm**

Yêu cầu

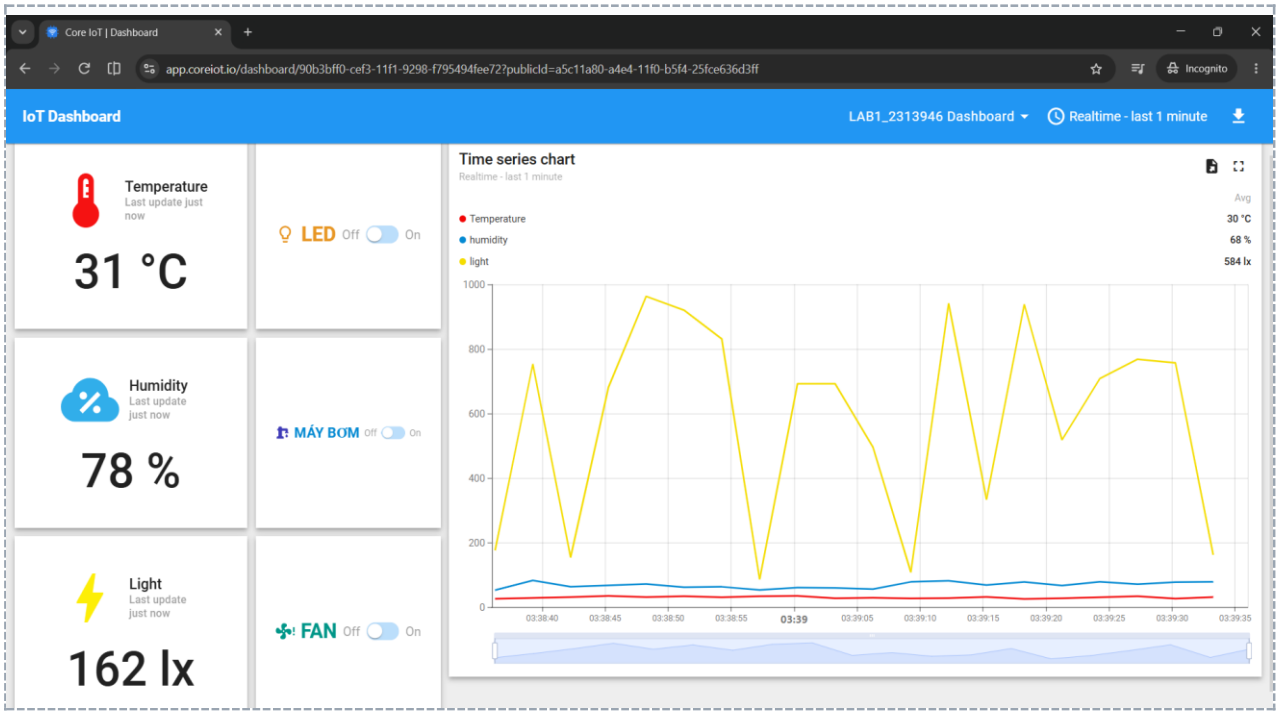
1. Đặt nhóm dashboard chứa dashboard của bạn ở chế độ **công khai** (public) và sao chép link công khai (Hướng dẫn 7).
2. Đặt nhóm thiết bị từ Bài tập 1 ở chế độ công khai.
3. Mở link trong cửa sổ trình duyệt **riêng tư / ẩn danh** (không đăng nhập) trong khi script đang chạy.
4. Ghi link vào bảng thông tin sinh viên ở trang 1.

Minh chứng

Ảnh 5.1. Hộp thoại **Public dashboard link**.



Ảnh 5.2. Dashboard được mở trong cửa sổ riêng tư, hiển thị dữ liệu trực tiếp.



Câu hỏi 5. Vì sao nhóm thiết bị cũng phải ở chế độ công khai? Điều gì có thể xảy ra nếu bạn đặt mọi dashboard và thiết bị trong tài khoản của mình ở chế độ công khai?

> **- Vì sao nhóm thiết bị phải công khai:** Trên CoreIOT, Dashboard chỉ đóng vai trò là tầng giao diện trực quan hóa, còn quyền truy cập dữ liệu được quản lý ở tầng thực thể (Device). Khi một người dùng ẩn danh mở Dashboard công khai, nếu thiết bị vẫn ở chế độ riêng tư (Private) thì họ không có quyền đọc telemetry từ thiết bị đó, khiến các widget bị lỗi hoặc hiển thị trắng (*No data*).
>
> **- Rủi ro khi public mọi thứ:** Toàn bộ dữ liệu nhạy cảm, lịch sử đo đạc và thông tin nội bộ của hệ thống sẽ bị phơi bày cho bất kỳ ai trên Internet. Đặc biệt, nếu các thiết bị chấp hành và công tắc điều khiển cũng bị public, kẻ xấu có thể tùy tiện kích hoạt bật/tắt thiết bị vật lý từ xa, dẫn đến nguy cơ hỏng hóc thiết bị, cháy nổ hoặc mất an toàn lao động trong môi trường công nghiệp.

Trước khi nộp bài

- ☐ Toàn bộ thông tin sinh viên ở trang 1 đã được điền, bao gồm **link dashboard công khai**.
- ☐ Mọi khung nét đứt đều chứa ảnh chụp màn hình rõ ràng; tên thiết bị của bạn hiển thị trong mỗi bài tập.
- ☐ Các khung mã chứa mã của bạn với thụt lề đúng.
- ☐ Access token đã được **xóa hoặc làm mờ** ở mọi nơi.
- ☐ Đã trả lời đủ cả năm câu hỏi.
- ☐ File được lưu với tên `LAB1_<StudentID>.docx` (hoặc xuất ra PDF nếu giảng viên yêu cầu).

Thang điểm

Bài tập	Đạt điểm tối đa khi…	Điểm
1	Thiết bị, token tùy chỉnh và nhóm đúng và hiển thị rõ (1,0); câu trả lời đúng (0,5).	1.5
2	Mã thêm `light` và chu kỳ 3 s đúng (1,0); dữ liệu hiển thị trong Latest telemetry (0,5); câu trả lời (0,5).	2.0
3	Ba thẻ được gắn nhãn rõ ràng (1,0); biểu đồ có ba series (1,0); câu trả lời (0,5).	2.5
4	Ba công tắc hoạt động (1,0); mã xử lý FAN và in ra thay đổi trạng thái (1,0); câu trả lời (0,5).	2.5
5	Link công khai hoạt động trong cửa sổ riêng tư (1,0); câu trả lời (0,5).	1.5
	Tổng	**10**